

Lab5 实验报告

姓名：朱文凯

学号：23307110192

1. 实验目标

本次 Lab5 的目标是在前面已经完成 CSR 和基础流水线控制的基础上，继续支持 RISC-V 特权级切换和 Sv39 地址翻译，使 CPU 能够运行带用户态程序的测试内核。

根据实验要求，本次主要完成：

- 支持 ECALL，在用户态或机器态触发同步异常
- 支持 MRET，从机器态 trap 返回到 mstatus.MPP 指定的特权级
- 维护当前 CPU 特权级，并接入 DiffTest
- 使用 mepc、mtvec、mcause、mstatus 完成基本 trap 上下文切换
- 实现 Sv39 MMU，在 U/S 模式且 satp.mode == 8 时完成虚拟地址到物理地址的转换
- 让取指和访存都经过统一的地址翻译路径

2. 总体设计思路

这次实现没有重写已有五级流水线，而是在原有结构上增加两条主线：

第一条是特权级和 trap 控制。ECALL 和 MRET 在译码阶段被识别出来，但真正修改 CSR、切换特权级和重定向 PC 都放在 WB 提交边界完成。这样可以保证异常处理是精确的：异常指令之前的指令已经提交，异常指令之后的错误路径指令会被 flush。

第二条是地址翻译。原来的取指和访存分别通过 IBusToCbus、DBusToCbus 接入 CbusArbiter。本次采用实验要求中的方式 2，在仲裁后的 Cbus 上插入统一 MMU。这样取指和访存都会走同一个翻译模块，也避免了分别维护两套页表遍历逻辑。

整体数据通路如下：

```
Core
├─ IBus -> IBusToCbus ┐
│                       │
└─ DBus -> DBusToCbus ┘
                        │
                        └─ CbusArbiter -> MMU -> RAM / MMIO
```

core 额外向顶层导出当前 `priv_mode` 和 `satp`，供 MMU 判断是否需要启用 Sv39 翻译。

3. 主要实现内容

3.1 ECALL / MRET 译码

在 `decode_out_t` 中增加 `is_ecall` 和 `is_mret`，并在 `core_decode.sv` 中识别：

- `32'h00000073` : ECALL
- `32'h30200073` : MRET

这两条指令不写通用寄存器，也不走普通 CSR 指令写回路径。普通 `CSRRW`、`CSRRS`、`CSRRC` 仍沿用 Lab4 的处理方式。

3.2 trap 与特权级更新

CPU 新增当前特权级寄存器，上电复位后为 M 模式。Difftest 中的 `privilegeMode` 不再固定为 `2'd3`，而是接入真实特权级。

执行 ECALL 时，硬件完成：

- `mepc` <- 当前 ECALL 指令 PC
- `mcause` <- 8 或 11，分别表示来自 U 模式或 M 模式的环境调用
- `mstatus.MPIE` <- `mstatus.MIE`
- `mstatus.MIE` <- 0
- `mstatus.MPP` <- trap 前的特权级
- 当前特权级切换到 M 模式
- PC 重定向到 `mtvec`

执行 MRET 时，硬件完成：

- PC 重定向到 `mepc`
- 当前特权级恢复为 `mstatus.MPP`
- `mstatus.MIE` <- `mstatus.MPIE`
- `mstatus.MPIE` <- 1
- `mstatus.MPP` <- U
- 当返回到低于 M 模式的特权级时，清除 `MPRV`

这些动作在 WB 提交边界触发，并同步 `flush` 前面已经取到的错误路径指令。

3.3 Sv39 MMU

本次新增 `MMU.sv`，位于仲裁后的 CBus 上。启用条件严格按实验要求：

- 当前为 M 模式：不启用 MMU，直接旁路
- `satp.mode == 0`：不启用 MMU，直接旁路
- 当前为 U/S 模式且 `satp.mode == 8`：启用 Sv39 翻译

MMU 使用状态机进行页表遍历。根页表物理地址来自 `satp.ppn << 12`，三级索引分别使用虚拟地址的 `[38:30]`、`[29:21]`、`[20:12]`。每一级读取 64 位 PTE，如果遇到叶子 PTE，则根据当前层级拼接物理地址；如果不是叶子，则继续访问下一级页表。

除了实验要求的固定三级页表，本次实现也顺带支持了上级页表项直接作为叶子项的情况，也就是 2MB 或 1GB 大页映射。

3.4 仿真测试镜像处理

仓库中的 Lab5 目录提供了 `kernel.coe`，但 Makefile 的 `test-lab5` 目标需要 `kernel.bin`。因此本地先按 `bin2coe.py` 的字节序规则从 `kernel.coe` 恢复出 `kernel.bin`，并补齐 BSS 区的零初始化内容。

为了让 Diffptest 参考模型与本次软件环境一致，本地仿真镜像还补充了：

- 入口处 PMP 全开放初始化，避免参考模型在进入 U 模式后因 PMP 默认关闭而报访问异常
- 叶子 PTE 的 A/D 位，使参考模型不会因 `accessed/dirty` 位为 0 而触发页错误

这些调整只影响本地仿真测试镜像，不改变 CPU RTL 的功能设计。

4. 关键问题与处理

4.1 ECALL / MRET 不能当作普通 CSR 指令

一开始如果把 `ECALL`、`MRET` 当成 `SYSTEM funct3=000` 的普通空操作，程序只能在 M 模式下继续顺序执行，无法真正进入用户态，也无法从用户态返回内核。

因此这两条指令需要独立控制信号，并且不能复用普通 CSR 指令的写回路径。普通 CSR 写回只修改一个 CSR，而 `trap / mret` 需要同时修改多个 CSR、切换特权级并重定向 PC。

4.2 trap 重定向必须和流水线 flush 绑定

ECALL 和 MRET 都会改变后续取指方向。如果只修改 PC，而不清理已经进入流水线的年轻指令，就可能出现错误路径指令继续提交的问题。

本次选择在 WB 提交边界统一触发重定向，并清空 IF/ID、ID/EX、EX/MEM、MEM/WB 中的无效路径状态。这样做性能不是最优，但实现直接，且更容易保证精确异常语义。

4.3 MMU 需要统一处理取指和访存

Lab5 的用户程序在 U 模式下运行，取指地址和数据访存地址都是虚拟地址。如果只翻译数据访存，用户态第一条指令就无法正确获取；如果只翻译取指，后续系统调用保存上下文也会出错。

因此 MMU 放在 CBus 仲裁之后，由取指和访存共享。这样只需要一套页表 walk 状态机，也更符合实验指导中“单一 MMU”的要求。

4.4 仿真镜像和上板镜像必须保持一致

调试过程中曾出现过一个比较容易忽略的问题：Verilator 仿真使用的是 kernel.bin，而 Vivado BRAM 初始化使用的是 kernel.coe。最初我只修复并补齐了本地仿真使用的 kernel.bin，包括 BSS 补零、PMP 初始化和 PTE A/D 位兼容处理，但上板使用的 kernel.coe 仍是旧内容。

这会导致仿真已经能输出 Return from init! Test passed，但上板仍然卡在 userinit ok 之后。后续通过重新用 bin2coe.py 从通过仿真的 kernel.bin 生成 kernel.coe，并在 Vivado 中重新生成 BRAM IP 的 output products，才保证板端运行的程序和仿真程序一致。

这个问题说明，上板调试时不能只看 RTL 是否更新，还要确认片上存储器的初始化文件是否同步更新。

4.5 MMU 需要适配板端 ready 时序

另一个上板阶段暴露的问题是 MMU 与板端 BRAM 访问时序的配合。仿真 RAM 的响应比较理想，而 Vivado 板级路径中的 BRAM wrapper 有固定访问延迟，且 ready/last 相对请求可能多保持或延后一拍。

原来的 MMU 状态机在页表 walk 时，只要看到 oresp.ready && oresp.last，就立即进入下一级页表访问或发起最终访存请求。这样在连续三级页表 walk 中，上一笔请求的响应有可能被误认为下一笔请求的响应；在最后一级 PTE 之后，也可能把页表项响应和真正的用户态取指响应混在一起。现象上表现为：串口已经打印到 userinit ok，但执行 mret 后无法正确运行用户态 initcode，因此没有最后一句输出。

修复方法是在 MMU 中增加一个 `STATE_WAIT_CLEAR` 状态。每次完成一次 PTE 读取或最终访存后，先插入一个空拍，将 `oreq` 和 `iresp` 拉低，再进入下一次请求。这样可以把连续请求之间的响应边界隔开，避免板端 `ready` 多一拍带来的误判。需要注意的是，这里不能简单等待 `ready` 变低，因为板级外设路径在 `valid=0` 时也可能让 `ready` 保持为 1，直接等待低电平反而可能造成死锁。

5. 仿真结果

Test Lab5:

```
make test-lab5
```

关键输出为：

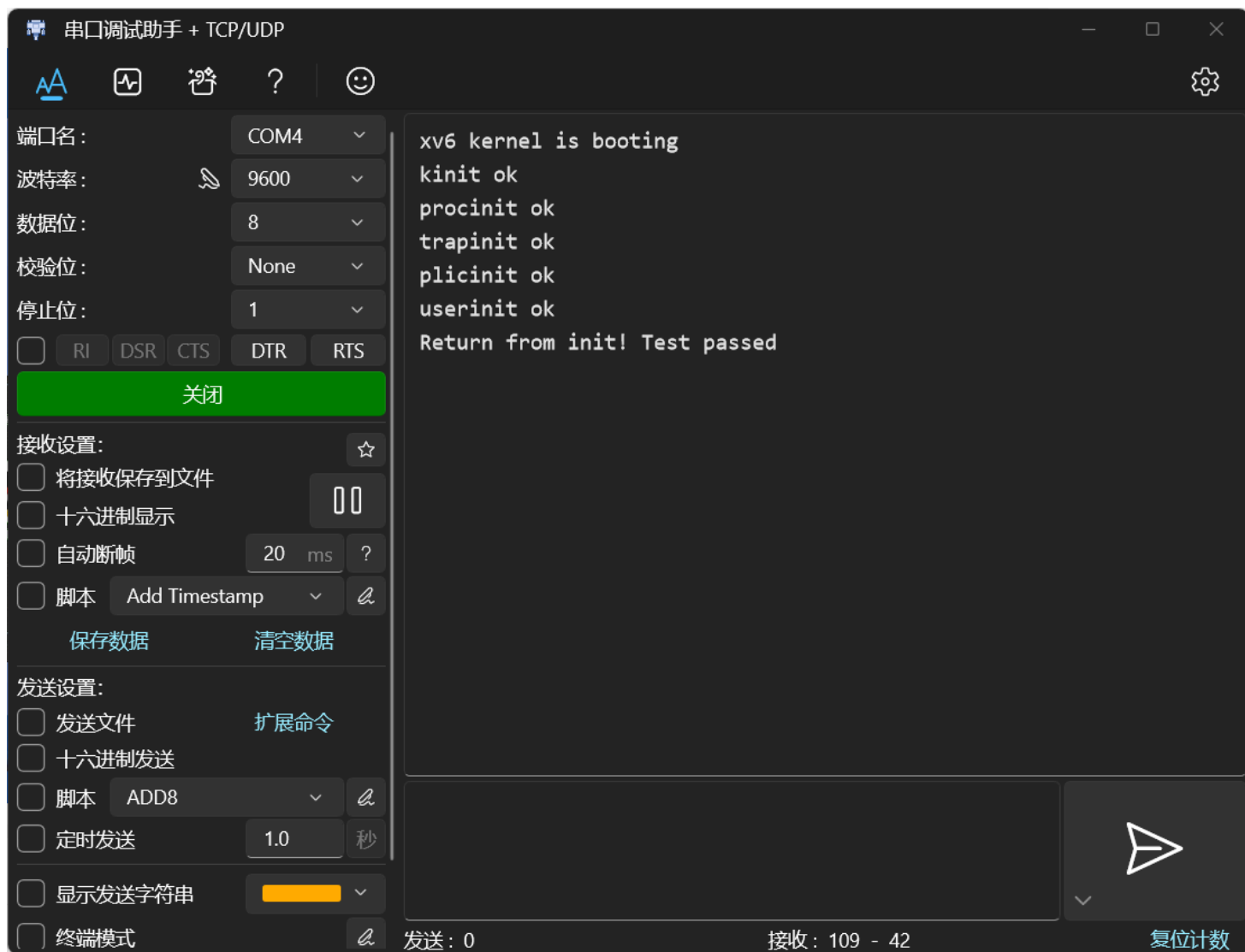
```
The first instruction of core 0 has committed. Dfftest enabled.
xv6 kernel is booting
kinit ok
procinit ok
trapinit ok
plicinit ok
userinit ok
Return from init! Test passed
^CCore 0: SOME SIGNAL STOPS THE PROGRAM at pc = 0x7ffff000c
total guest instructions = 336,504
instrCnt = 336,504, cycleCnt = 4,740,965, IPC = 0.070978
Seed=0 Guest cycle spent: 4,740,967 (this will be different from cycleCnt if emu loads a snapshot)
Host time spent: 2,020ms
This emulator compiled with JTAG Remote Bitbang client. To enable, use +jtag_rbb_enable=1.
Listening on port 23334
make: *** [Makefile:59: test-lab5] 中断
```

根据实验要求，Lab5 最后卡住是正常现象。出现 `Return from init! Test passed` 说明仿真部分已经通过。

6. 上板验证

本次 Lab5 要求实际上板，通过串口查看输出。由于上板测试需要在之后实际连接 FPGA 开发板和串口后完成，这一部分先预留材料位置。

待补充内容：



符合预期测试的串口输出。

7. 总结

Lab5 相比前几次实验，重点从普通指令执行扩展到了“CPU 当前处于什么特权级、访问的地址是否需要翻译、异常发生时架构状态如何保持一致”。

这次实现中我体会比较明显的是，特权级切换和 MMU 不能只看单条指令本身，还要和流水线提交、CSR 可见性、DiffTest 状态以及总线访问时序一起考虑。尤其是 MRET 之后第一条用户态取指，能很好地检验 mepc、mstatus.MPP、satp、MMU 和 flush 是否真正配合正确。

8. 大模型的使用说明

本次实验中，我使用了 Cursor / Gemini 辅助完成以下工作：

- 梳理 Lab5 原始要求和本仓库已有 Lab4 基础

- 分析 ECALL、MRET、mstatus 字段更新顺序
- 辅助定位 Difftest 中的 CSR、特权级和 MMU 相关失败点
- 协助整理报告结构和表述

核心 RTL 修改、测试结果判断和最终实现取舍均结合代码、波形/日志信息和实验要求进行人工确认。